Cloud-containerization-and-DevOps

Experiment 4: Docker Essentials

`Dockerfile` `.dockerignore` `tagging` `publishing`

Containerizing Applications with Dockerfile

Step-1: Create a Directory for Python application

```
mkdir my-flask-app
cd my-flask-app
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4$ mkdir my-flask-app
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4$ cd my-flask-app
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-2:- Create an ** `app.py` :**

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
palak@LAPTOP-2SDBVCPO: /
  GNU nano 7.2                    app.py *
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step-3:- Create ** `requirements.txt` :**

```
Flask==2.3.3
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ nano requirements.tx
t
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ cat requirements.txt
Flask==2.3.3
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-4: Create **Dockerfile**

```dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ nano Dockerfile
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ cat Dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-5:- Create a ** `.dockerignore` :**

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
```

```
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ nano .dockerignore
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ cat .dockerignore
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Why ***`.dockerignore`*** *is Important*

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4$ cd my-flask-app
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker build -t my-f
lask-app .
[+] Building 20.3s (10/10) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                    0.2s
 => => transferring dockerfile: 361B                                    0.1s
 => [internal] load metadata for docker.io/library/python:3.9-slim      1.9s
 => [internal] load .dockerignore                                       0.1s
 => => transferring context: 273B                                       0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b   0.1s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b   0.1s
 => [internal] load build context                                       0.1s
 => => transferring context: 321B                                       0.1s
 => CACHED [2/5] WORKDIR /app                                           0.0s
 => [3/5] COPY requirements.txt .                                       0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt           13.2s
 => [5/5] COPY app.py .                                                 0.2s
 => exporting to image                                                  4.0s
 => => exporting layers                                                 2.2s
 => => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978   0.0s
 => => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128   0.0s
 => => exporting attestation manifest sha256:d5af0b7ca2d416dcc86c7d82   0.1s
 => => exporting manifest list sha256:4031a16a4a75b78cba7b150fabf3628   0.0s
 => => naming to docker.io/library/my-flask-app:latest                  0.1s
 => => unpacking to docker.io/library/my-flask-app:latest               1.3s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-7:- Verify Image

```
docker images
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker images
REPOSITORY                TAG       IMAGE ID       CREATED          SIZE
my-flask-app              latest    4031a16a4a75   31 seconds ago   200MB
nginx_alpine_custom       latest    e1920b54b1d3   7 days ago       92.5MB
nginx_ubuntu_custom       latest    bd03b4cee3df   10 days ago      226MB
nginx_latest_custom       latest    60cfe9a9a0c0   10 days ago      237MB
myimage                   1.0       6294b3435f74   2 weeks ago      183MB
palakagarwal16/java-app   1.0       6294b3435f74   2 weeks ago      183MB
mysql                     latest    db32c8ec843c   2 weeks ago      1.27GB
nginx                     latest    341bf0f3ce6c   2 weeks ago      240MB
alpine                    latest    25109184c71b   3 weeks ago      13.1MB
ubuntu                    latest    cd1dba651b30   5 weeks ago      119MB
busybox                   latest    b3255e7dfbcd   17 months ago    6.77MB
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker build -t my-f
lask-app:1.0 .
[+] Building 2.3s (10/10) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                    0.1s
 => => transferring dockerfile: 361B                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim      1.4s
 => [internal] load .dockerignore                                       0.1s
 => => transferring context: 273B                                       0.1s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b   0.1s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b   0.1s
 => [internal] load build context                                       0.1s
 => => transferring context: 63B                                        0.1s
 => CACHED [2/5] WORKDIR /app                                           0.0s
 => CACHED [3/5] COPY requirements.txt .                                0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt     0.0s
 => CACHED [5/5] COPY app.py .                                          0.0s
 => exporting to image                                                  0.3s
 => => exporting layers                                                 0.0s
 => => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978   0.0s
 => => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128   0.0s
 => => exporting attestation manifest sha256:f813f338c370437fbb6214d0   0.1s
 => => exporting manifest list sha256:84617a456e6cd533d724e729969c72a   0.0s
 => => naming to docker.io/library/my-flask-app:1.0                     0.0s
 => => unpacking to docker.io/library/my-flask-app:1.0                  0.0s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

Step-12: View Image Details

```
docker images
```

Step-13:- Show image history

```
docker history my-flask-app
```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

Step-16:- Test the application

```
curl http://localhost:5000
```

Step-17:- View running containers

```
docker ps
```

Step-18:- View container logs

```
docker logs flask-container
```

Step-19:- Stop container

```
docker stop flask-container
```

Step-20:- Start stopped container

```
docker start flask-container
```

Step-21:- Remove container

```
docker rm flask-container
```

Step-22:- If working then Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple ** `Dockerfile.multistage` :**

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt
```

```
# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

Step-24:- Build regular image

```
docker build -t flask-regular .
```

Step-25:- Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

Step-26:- Compare sizes

```
docker images | grep flask-
```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

Step-31:- Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app
cd my-node-app
```

Step-34:- Create a ** `app.js` :**

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
    res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
    res.json({ status: 'healthy' });
});

app.listen(port, () => {
    console.log(`Server running on port ${port}`);
});
```

Step-35:- Create a ** `package.json` :**

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step-36:- Create a Node.js Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000
```

```
CMD ["node", "app.js"]
```

Step-37:- Build image

```
docker build -t my-node-app .
```

Step-38:- Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

Step-40:- Test on Browser

DockerHub

View Pushed Image on DockerHub

Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
`docker build`	Build image	`docker build -t myapp .`
`docker run`	Run container	`docker run -p 3000:3000 myapp`

Command	Purpose	Example
`docker ps`	List containers	`docker ps -a`
`docker images`	List images	`docker images`
`docker tag`	Tag image	`docker tag myapp:latest myapp:v1`
`docker login`	Login to Dockerhub using username and password or token	`echo "token" | docker login -u username --password-stdin`
`docker push`	Push to registry	`docker push username/myapp`
`docker pull`	Pull from registry	`docker pull username/myapp`
`docker rm`	Remove container	`docker rm container-name`
`docker rmi`	Remove image	`docker rmi image-name`
`docker logs`	View logs	`docker logs container-name`
`docker exec`	Execute command	`docker exec -it container-name bash`

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
2. **.dockerignore** excludes unnecessary files (important for security and performance)
3. **Tagging** helps version and organize images
4. **Multi-stage builds** create smaller, more secure production images
5. **Docker Hub** allows sharing and distributing images
6. **Always test** images locally before publishing

Cleanup

```
# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove everything unused
docker system prune -a
```